Flexbox

1. Что такое Flexbox. Flex Container
2. Направление flex-direction
3. flex-wrap
4. flex-flow. Порядок элементов
5. Выравнивание элементов. justify-content
6. Выравнивание элементов. align-items и align-self
7. Выравнивание строк и столбцов. align-content
8. Управление элементами. flex-basis, flex-shrink и flex-grow
9. Многоколоночный дизайн на Flexbox
10. Макет страницы на Flexbox

Что такое Flexbox. Flex Container

Flexbox - это общее название для модуля **Flexible Box Layout**, который имеется в CSS3. Данный модуль определяет особый режим компоновки/верстки пользовательского интерфейса, который называется **flex layout**.

В этом плане Flexbox предоставляет иной подход к созданию пользовательского интерфейса, который отличается от табличной или блочной верстки.

Благодаря Flexbox проще создавать сложные, комплексные интерфейсы, где с легкостью можем переопределять направление и выравнивание элементов, создавать адаптивные табличные представления.

Кроме того, Flexbox довольно прост в использовании.

Единственная проблема, которая может возникнуть при его применении, - это кроссбраузерность. Например, в Internet Explorer поддержка Flexbox и то частичная появилась только в последней версии - IE11. В то же время все современные браузеры, в том числе Microsoft Edge, Opera, Google Chrome, Safari, Firefox, имеют полную поддержку данного модуля.

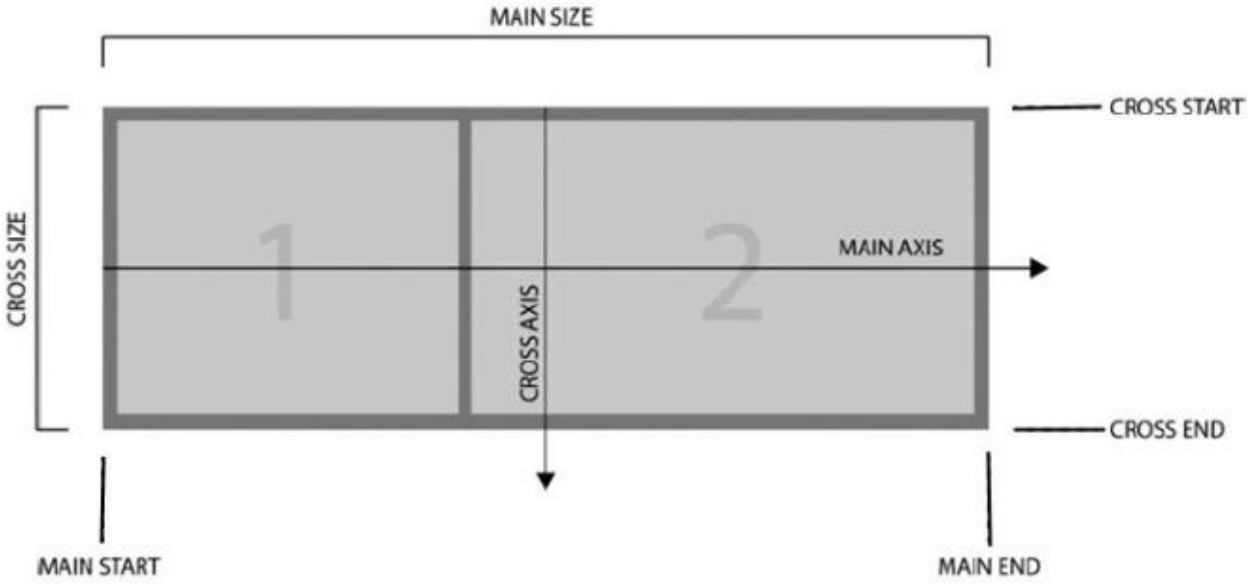
Основными составляющими компоновки flexbox являются flex-контейнер (flex container) и flex-элементы (flex items).

Flex container представляет некоторый элемент, внутри которого размещены flex-элементы.

Основные понятия

Прежде чем переходить к изучению верстки flexbox, стоит рассмотреть некоторые основные понятия.

Одно из ключевых понятий представляет **main axis** или центральная ось. Это условная ось во flex-контейнере, вдоль которой позиционируются flex-элементы.


MAIN SIZE
CROSS START
CROSS SIZE
1
2
MAIN AXIS
CROSS AXIS
CROSS END
MAIN START
MAIN END

Элементы в контейнере могут располагаться по горизонтали в виде строки и по вертикали в виде столбца. В зависимости от типа расположения будет меняться и центральная ось.

Если расположение в виде строки, то центральная ось направлена горизонтально слева направо.

Если расположение в виде столбца, то центральная ось направлена вертикально сверху вниз.

Термины **main start** и **main end** описывают соответственно начало и конец центральной оси, а расстояние между между ними обозначается как **main size**.

Кроме основной оси существует также поперечная ось или **cross axis**. Она перпендикулярна основной. При расположении элементов в виде строки **cross axis** направлена сверху вниз, а при расположении в виде столбца она направлена слева направо. Начало поперечной оси обозначается как **cross start**, а ее конец - как **cross end**. Расстояние между ними описывается термином **cross size**.

То есть, если элементы располагаются в строку, то **main size** будет представлять ширину контейнера или элементов, а **cross size** - их высоту.

Если же элементы располагаются в столбик, то, наоборот, **main size** представляет высоту контейнера и элементов, а **cross size** - их ширину.

Создание flex-контейнера

Для создания flex-контейнера необходимо присвоить его стилевому свойству **display** одно из двух значений: **flex** или **inline-flex**.

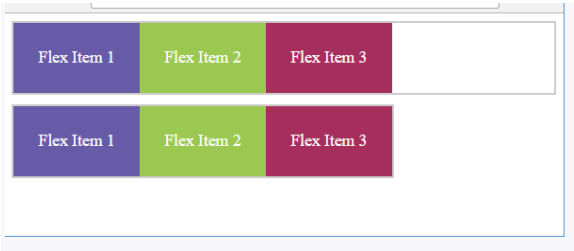
Создадим простейшую веб-страницу, которая применяет flexbox:

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>
            .flex-container {
                display: flex;
            }
            .flex-item {
                text-align:center;
                font-size: 1.1em;
                padding: 1.5em;
                color: white;
            }
            .color1 {background-color: #675BA7;}
            .color2 {background-color: #9BC850;}
            .color3 {background-color: #A62E5C;}
        </style>
    </head>
```

```html
21     <body>
22         <div class="flex-container">
23             <div class="flex-item color1">Flex Item 1</div>
24             <div class="flex-item color2">Flex Item 2</div>
25             <div class="flex-item color3">Flex Item 3</div>
26         </div>
27     </body>
28 </html>
```

Если значение **flex** определяет контейнер как блочный элемент, то значение **inline-flex** определяет элемент как строчный (*inline*). Рассмотрим оба способа на примере:

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>
            .flex-container {
                display: flex;
                border:2px solid #ccc;
            }
            .inline-flex-container {
                display: inline-flex;
                border:2px solid #ccc;
                margin-top:10px;
            }
            .flex-item {
                text-align:center;
                font-size: 1.1em;
                padding: 1.5em;
                color: white;
            }
            .color1 {background-color: #675BA7;}
            .color2 {background-color: #9BC850;}
            .color3 {background-color: #A62E5C;}
        </style>
    </head>
```



В частности, в первом случае flex-контейнер растягивается по ширине страницы, а во втором случае занимает именно столько места, сколько необходимо для flex-элементов.

```html
    <body>
        <div class="flex-container">
            <div class="flex-item color1">Flex Item 1</div>
            <div class="flex-item color2">Flex Item 2</div>
            <div class="flex-item color3">Flex Item 3</div>
        </div>

        <div class="inline-flex-container">
            <div class="flex-item color1">Flex Item 1</div>
            <div class="flex-item color2">Flex Item 2</div>
            <div class="flex-item color3">Flex Item 3</div>
        </div>
    </body>
</html>
```

Направление flex-direction

Flex-элементы во flex-контейнере могут иметь определенное направление, а именно они могут располагаться в виде строк или в виде столбцов.

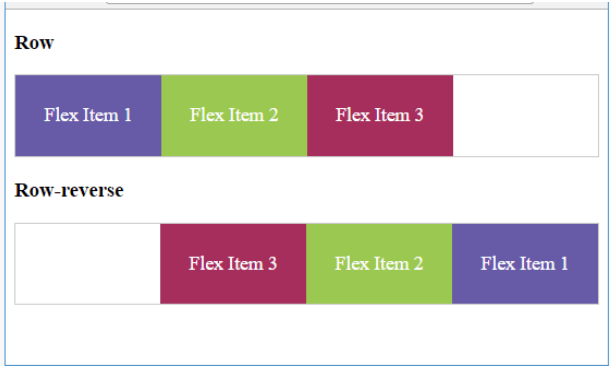
Для управления направлением элементов CSS3 предоставляет свойство **flex-direction**. Оно определяет направление элементов и может принимать следующие значения:

- **row**: значение по умолчанию, при котором элементы располагаются в виде строки слева направо
- **row-reverse**: элементы также располагаются в виде стоки только в обратном порядке справа налево
- **column**: элементы располагаются в столбик сверху вниз

- **column-reverse**: элементы располагаются в столбик в обратном порядке снизу вверх

Например, расположение в виде строки:

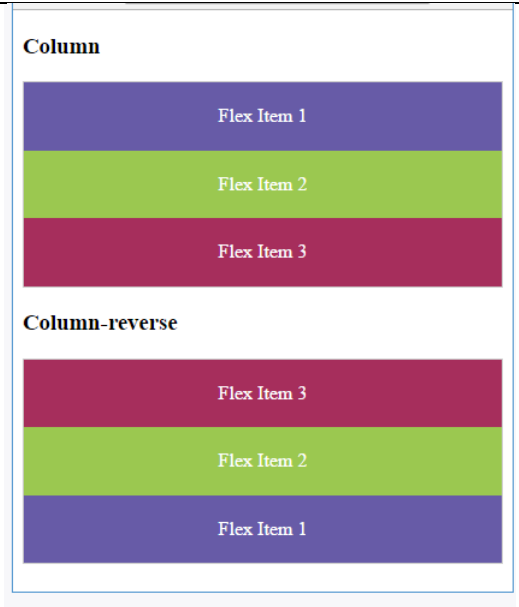
```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>
            .flex-container {
                display: flex;
                border:1px solid #ccc;
            }
            .row {
                flex-direction: row;
            }
            .row-reverse {
                flex-direction: row-reverse;
            }
            .flex-item {
                text-align:center;
                font-size: 1.1em;
                padding: 1.5em;
                color: white;
            }
            .color1 {background-color: #675BA7;}
            .color2 {background-color: #9BC850;}
            .color3 {background-color: #A62E5C;}
        </style>
    </head>
```



```html
28      <body>
29          <h3>Row</h3>
30          <div class="flex-container row">
31              <div class="flex-item color1">Flex Item 1</div>
32              <div class="flex-item color2">Flex Item 2</div>
33              <div class="flex-item color3">Flex Item 3</div>
34          </div>
35
36          <h3>Row-reverse</h3>
37          <div class="flex-container row-reverse">
38              <div class="flex-item color1">Flex Item 1</div>
39              <div class="flex-item color2">Flex Item 2</div>
40              <div class="flex-item color3">Flex Item 3</div>
41          </div>
42      </body>
43  </html>
```

Аналогично работает расположение в виде столбца:

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>
            .flex-container {
                display: flex;
                border:1px solid #ccc;
            }
            .column {
                flex-direction: column;
            }
            .column-reverse {
                flex-direction: column-reverse;
            }
            .flex-item {
                text-align:center;
                font-size: 1em;
                padding: 1.2em;
                color: white;
            }
            .color1 {background-color: #675BA7;}
            .color2 {background-color: #9BC850;}
            .color3 {background-color: #A62E5C;}
        </style>
    </head>
```



```html
    <body>
        <h3>Column</h3>
        <div class="flex-container column">
            <div class="flex-item color1">Flex Item 1</div>
            <div class="flex-item color2">Flex Item 2</div>
            <div class="flex-item color3">Flex Item 3</div>
        </div>

        <h3>Column-reverse</h3>
        <div class="flex-container column-reverse">
            <div class="flex-item color1">Flex Item 1</div>
            <div class="flex-item color2">Flex Item 2</div>
            <div class="flex-item color3">Flex Item 3</div>
        </div>
    </body>
</html>
```

flex-wrap

Свойство **flex-wrap** определяет, будет ли flex-контейнер несколько рядов элементов (строк или столбцов) в случае если его размеры недостаточны, чтобы вместить в один ряд все элементы. Это свойство может принимать следующие значения:

- **nowrap**: значение по умолчанию, которое определяет flex-контейнер, где все элементы раполагаются в одну строку (при расположении в виде строк) или один столбец (при расположении в столбик)

- **wrap**: если элементы не помещаются во flex-контейнер, то создает дополнительные ряды в контейнере для размещения элементов.

При расположении в виде строки создаются дополнительные строки, а при расположении в виде столбца добавляются дополнительные столбцы

- **wrap-reverse**: то же самое, что и значение wrap, только элементы располагаются в обратном порядке

Например, возьмем значение по умолчанию `nowrap`:

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>
            .flex-container {
                display: flex;
                border: solid 0.25em #000;
                width: 60%;
                height:8.25em;
                flex-wrap:nowrap;
            }
            .row {
                flex-direction: row;
            }
            .row-reverse {
                flex-direction: row-reverse;
            }
            .flex-item {
                text-align:center;
                font-size: 1em;
                padding: 1.5em;
                color: white;
                opacity: 0.8;
            }
            .color1 {background-color: #675BA7;}
            .color2 {background-color: #9BC850;}
            .color3 {background-color: #A62E5C;}
            .color4 {background-color: #2A9FBC;}
            .color5 {background-color: #F15B2A;}
        </style>
    </head>
```

Здесь в каждом из flex-контейнеров по пять элементов, однако ширина контейнера может вмещать не все элементы, тогда они уходят за границу контейнера

```html
34      <body>
35          <h3>Row</h3>
36          <div class="flex-container row">
37              <div class="flex-item color1">Flex Item 1</div>
38              <div class="flex-item color2">Flex Item 2</div>
39              <div class="flex-item color3">Flex Item 3</div>
40              <div class="flex-item color4">Flex Item 4</div>
41              <div class="flex-item color5">Flex Item 5</div>
42          </div>
43
44          <h3>Row-reverse</h3>
45          <div class="flex-container row-reverse">
46              <div class="flex-item color1">Flex Item 1</div>
47              <div class="flex-item color2">Flex Item 2</div>
48              <div class="flex-item color3">Flex Item 3</div>
49              <div class="flex-item color4">Flex Item 4</div>
50              <div class="flex-item color5">Flex Item 5</div>
51          </div>
52      </body>
53  </html>
```

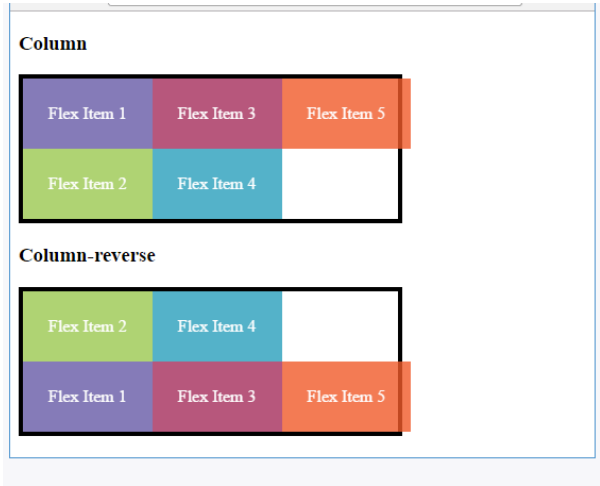
При установке значения `wrap` во flex-контейнере добавляются дополнительные ряды для помещения всех элементов в контейнере. Так, изменим значение свойства flex-wrap в контейнере:

```
.flex-container {
    display: flex;
    border: solid 0.25em #000;
    width: 60%;
    height:8.25em;

    flex-wrap:wrap;
}
```

В этом случае появится дополнительная стока:

При расположении в виде столбца контейнер будет создавать дополнительные столбцы:

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>

            .flex-container {
                display: flex;
                border: solid 0.25em #000;
                width: 60%;
                height:8.25em;
                flex-wrap:wrap;
            }
            .column {
                flex-direction: column;
            }
            .column-reverse {
                flex-direction: column-reverse;
            }
            .flex-item {
                text-align:center;
                font-size: 1em;
                padding: 1.5em;
                color: white;
                opacity: 0.8;
            }
            .color1 {background-color: #675BA7;}
            .color2 {background-color: #9BC850;}
            .color3 {background-color: #A62E5C;}
            .color4 {background-color: #2A9FBC;}
            .color5 {background-color: #F15B2A;}
        </style>
    </head>
```



```html
    <body>
        <h3>Column</h3>
        <div class="flex-container column">
            <div class="flex-item color1">Flex Item 1</div>
            <div class="flex-item color2">Flex Item 2</div>
            <div class="flex-item color3">Flex Item 3</div>
            <div class="flex-item color4">Flex Item 4</div>
            <div class="flex-item color5">Flex Item 5</div>
        </div>

        <h3>Column-reverse</h3>
        <div class="flex-container column-reverse">
            <div class="flex-item color1">Flex Item 1</div>
            <div class="flex-item color2">Flex Item 2</div>
            <div class="flex-item color3">Flex Item 3</div>
            <div class="flex-item color4">Flex Item 4</div>
            <div class="flex-item color5">Flex Item 5</div>
        </div>
    </body>
</html>
```

flex-flow. Порядок элементов

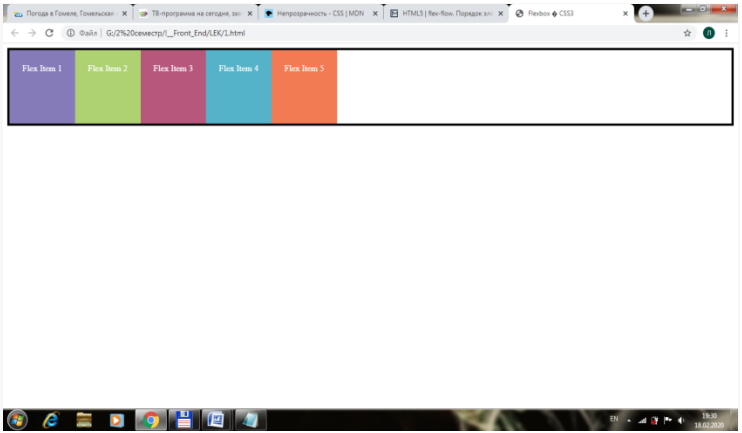
Свойство **flex-flow** позволяет установить значения сразу для обоих свойств `flex-direction` и `flex-wrap`. Оно имеет следующий формальный синтаксис:

```
flex-flow: [flex-direction] [flex-wrap]
```

Причем второе свойство - flex-wrap можно в принципе опустить, тогда для него будет использоваться значение по умолчанию - `nowrap`.

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>

            .flex-container {
                display: flex;
                border: solid 0.25em #000;
                height:8.25em;
                flex-flow: row wrap;
            }
            .flex-item {
                text-align:center;
                font-size: 1em;
                padding: 1.5em;
                color: white;
                opacity: 0.8;
            }
            .color1 {background-color: #675BA7;}
            .color2 {background-color: #9BC850;}
            .color3 {background-color: #A62E5C;}
            .color4 {background-color: #2A9FBC;}
            .color5 {background-color: #F15B2A;}
        </style>
    </head>
```

```html
28    <body>
29        <div class="flex-container">
30            <div class="flex-item color1">Flex Item 1</div>
31            <div class="flex-item color2">Flex Item 2</div>
32            <div class="flex-item color3">Flex Item 3</div>
33            <div class="flex-item color4">Flex Item 4</div>
34            <div class="flex-item color5">Flex Item 5</div>
35        </div>
36    </body>
37 </html>
```



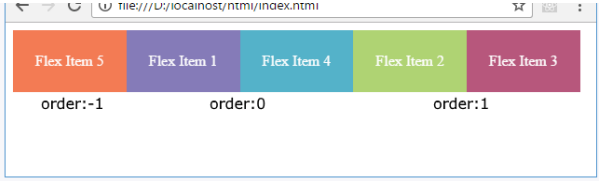
Свойство order

Свойство **order** позволяет установить группу для flex-элемента, позволяя тем самым переопределить его позицию внутри flex-контейнера. В качестве значения свойство принимает числовой порядок группы. К одной группе может принадлежать несколько элементов.

Например, элементы в группе 0 располагаются перед элементами с группой 1, а элементы с группой 1 располагаются перед элементами с группой 2 и так далее.

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>

            .flex-container {
                display: flex;
                flex-flow: row wrap;
            }
            .flex-item {
                text-align:center;
                font-size: 1em;
                padding: 1.5em;
                color: white;
                opacity: 0.8;
            }
            .group1{
                order:-1;
            }
            .group2{
                order:1;
            }
            .color1 {background-color: #675BA7;}
            .color2 {background-color: #9BC850;}
            .color3 {background-color: #A62E5C;}
            .color4 {background-color: #2A9FBC;}
            .color5 {background-color: #F15B2A;}
        </style>
    </head>
```

В данном случае определены 3 группы. Первый отображается последний элемент, так как он имеет группу -1



По умолчанию если у элементов явным образом не указано свойство order, то оно имеет значение 0. И последними в данном случае отображаются второй и третий элемент, так как у них свойство order равно 1.

```html
32    <body>
33        <div class="flex-container">
34            <div class="flex-item color1">Flex Item 1</div>
35            <div class="flex-item color2 group2">Flex Item 2</div>
36            <div class="flex-item color3 group2">Flex Item 3</div>
37            <div class="flex-item color4">Flex Item 4</div>
38            <div class="flex-item color5 group1">Flex Item 5</div>
39        </div>
40    </html>
```

Выравнивание элементов. justify-content

Иногда можем сталкиваться с тем, что пространство flex-контейнеров по размеру отличается от пространства, необходимого для flex-элементов. Например:

- flex-элементы не используют все пространство flex-контейнера

- flex-элементам требуется большее пространство, чем доступно во flex-контейнере. В этом случае элементы выходят за пределы контейнера.

Для управления этими ситуациями мы можем применять свойство **justify-content**. Оно выравнивает элементы вдоль основной оси - main axis (при расположении в виде строки по горизонтали, при расположении в виде столбца - по вертикали) и принимает следующие значения:

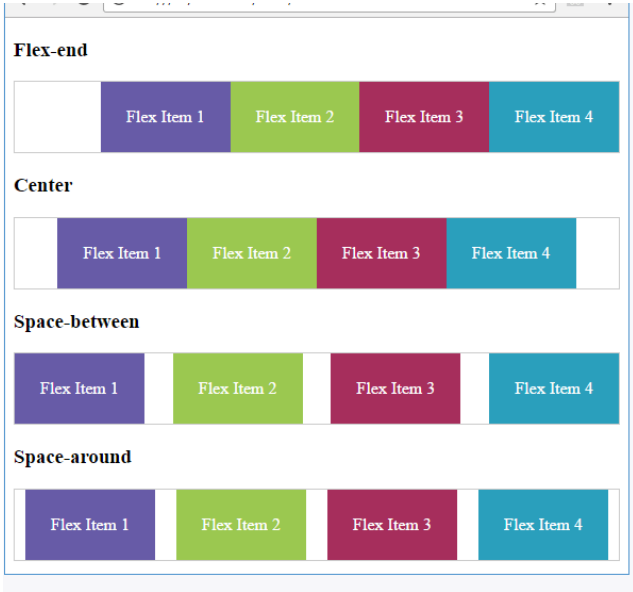
- **flex-start**: значение по умолчанию, при котором первый элемент выравнивается по левому краю контейнера(при расположении в виде строки) или по верху (при расположении в виде столбца), за ним располагается второй элемент и так далее.

- **flex-end**: последний элемент выравнивается по правому краю (при расположении в виде строки) или по низу (при расположении в виде столбца) контейнера, за ним выравнивается предпоследний элемент и так далее
- **center**: элементы выравниваются по центру
- **space-between**: если в стоке только один элемент или элементы выходят за границы flex-контейнера, то данное значение аналогично `flex-start`. В остальных случаях первый элемент выравнивается по левому краю (при расположении в виде строки) или по верху (при расположении в виде столбца), а последний элемент - по правому краю контейнера (при расположении в виде строки) или по низу (при расположении в виде столбца). Все оставшееся пространство между ними равным образом распределяется между остальными элементами

- **space-around**: если в строке только один элемент или элементы выходят за пределы контейнера, то его действие аналогично значению `center`. В ином случае элементы равным образом распределяют пространство между левым и правым краем контейнера, а расстояние между первым и последним элементом и границами контейнера составляет половину расстояния между элементами.

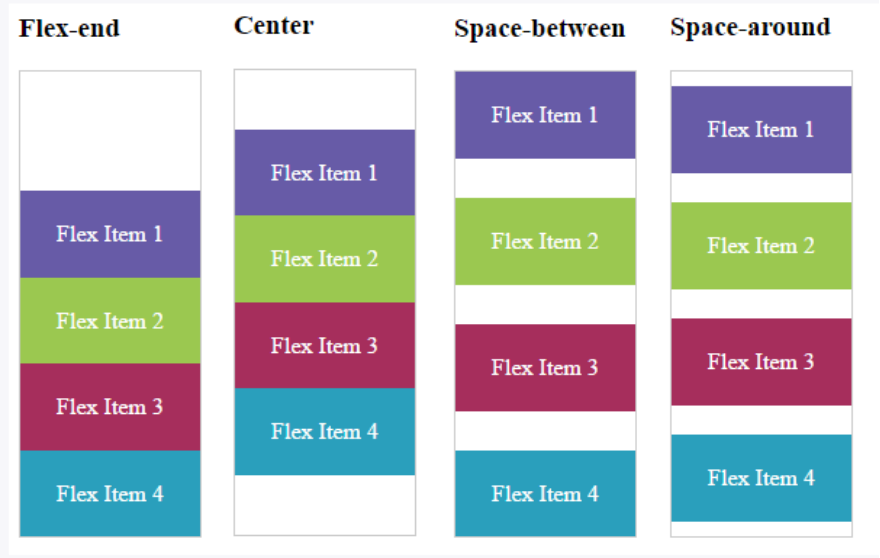
Выравнивание для расположения элементов в виде строки:

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>

            .flex-container {
                display: flex;
                border:1px #ccc solid;
            }
            .flex-end{
                justify-content: flex-end;
            }
            .center{
                justify-content: center;
            }
            .space-between{
                justify-content: space-between;
            }
            .space-around{
                justify-content: space-around;
            }
            .flex-item {
                text-align:center;
                font-size: 1em;
                padding: 1.5em;
                color: white;
            }
            .color1 {background-color: #675BA7;}
            .color2 {background-color: #9BC850;}
            .color3 {background-color: #A62E5C;}
            .color4 {background-color: #2A9FBC;}
            .color5 {background-color: #F15B2A;}
        </style>
    </head>
```



```html
    <body>
        <h3>Flex-end</h3>
        <div class="flex-container flex-end">
            <div class="flex-item color1">Flex Item 1</div>
            <div class="flex-item color2">Flex Item 2</div>
            <div class="flex-item color3">Flex Item 3</div>
            <div class="flex-item color4">Flex Item 4</div>
        </div>
        <h3>Center</h3>
        <div class="flex-container center">
            <div class="flex-item color1">Flex Item 1</div>
            <div class="flex-item color2">Flex Item 2</div>
            <div class="flex-item color3">Flex Item 3</div>
            <div class="flex-item color4">Flex Item 4</div>
        </div>
        <h3>Space-between</h3>
        <div class="flex-container space-between">
            <div class="flex-item color1">Flex Item 1</div>
            <div class="flex-item color2">Flex Item 2</div>
            <div class="flex-item color3">Flex Item 3</div>
            <div class="flex-item color4">Flex Item 4</div>
        </div>
        <h3>Space-around</h3>
        <div class="flex-container space-around">
            <div class="flex-item color1">Flex Item 1</div>
            <div class="flex-item color2">Flex Item 2</div>
            <div class="flex-item color3">Flex Item 3</div>
            <div class="flex-item color4">Flex Item 4</div>
        </div>
    </body>
</html>
```

Выравнивание при расположении в виде столбцов:



Выравнивание элементов. align-items и align-self

Свойство **align-items** также выравнивает элементы, но уже по поперечной оси (cross axis) (при расположении в виде строки по вертикали, при расположении в виде столбца - по горизонтали). Это свойство может принимать следующие значения:

- **stretch**: значение по умолчанию, при котором flex-элементы растягиваются по всей высоте (при расположении в строку) или по всей ширине (при расположении в столбик) flex-контейнера

- **flex-start**: элементы выравниваются по верхнему краю (при расположении в строку) или по левому краю (при расположении в столбик) flex-контейнера
- **flex-end**: элементы выравниваются по нижнему краю (при расположении в строку) или по правому краю (при расположении в столбик) flex-контейнера
- **center**: элементы выравниваются по центру flex-контейнера
- **baseline**: элементы выравниваются в соответствии со своей базовой линией

Выравнивание при расположении в строку:

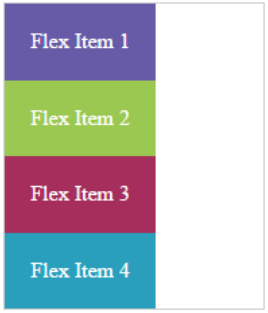
```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>

            .flex-container {
                display: flex;
                border:1px #ccc solid;
                height:5em;
            }
            .flex-start{
                align-items: flex-start;
            }
            .flex-end{
                align-items: flex-end;
            }
            .center{
                align-items: center;
            }
            .baseline{
                align-items: baseline;
            }
            .flex-item {
                text-align:center;
                font-size: 1em;
                padding: 1.2em;
                color: white;

            }
            .largest-item{
                padding-top:2em;
            }
            .color1 {background-color: #675BA7;}
            .color2 {background-color: #9BC850;}
            .color3 {background-color: #A62E5C;}
            .color4 {background-color: #2A9FBC;}
        </style>
    </head>
```

```html
    <body>
        <h3>Flex-start</h3>
        <div class="flex-container flex-start">
            <div class="flex-item color1">Flex Item 1</div>
            <div class="flex-item color2">Flex Item 2</div>
            <div class="flex-item color3">Flex Item 3</div>
            <div class="flex-item color4">Flex Item 4</div>
        </div>
        <h3>Flex-end</h3>
        <div class="flex-container flex-end">
            <div class="flex-item color1">Flex Item 1</div>
            <div class="flex-item color2">Flex Item 2</div>
            <div class="flex-item color3">Flex Item 3</div>
            <div class="flex-item color4">Flex Item 4</div>
        </div>
        <h3>Center</h3>
        <div class="flex-container center">
            <div class="flex-item color1">Flex Item 1</div>
            <div class="flex-item color2">Flex Item 2</div>
            <div class="flex-item color3">Flex Item 3</div>
            <div class="flex-item color4">Flex Item 4</div>
        </div>
        <h3>Baseline</h3>
        <div class="flex-container baseline">
            <div class="flex-item color1">Flex Item 1</div>
            <div class="flex-item color2 largest-item">Flex Item 2</div>
            <div class="flex-item color3">Flex Item 3</div>
            <div class="flex-item color4">Flex Item 4</div>
        </div>
</html>
```

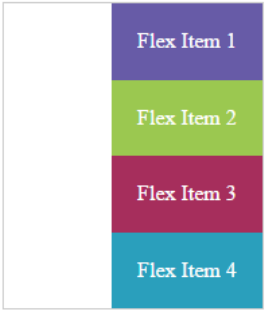
Аналогично свойство работает при расположении в столбик. Например, изменим стили flex-контейнера следующим образом:

```
.flex-container {
    display: flex;
    border:1px #ccc solid;
    flex-direction:column;
    width:12em;
}
```

Flex-start



Flex-end



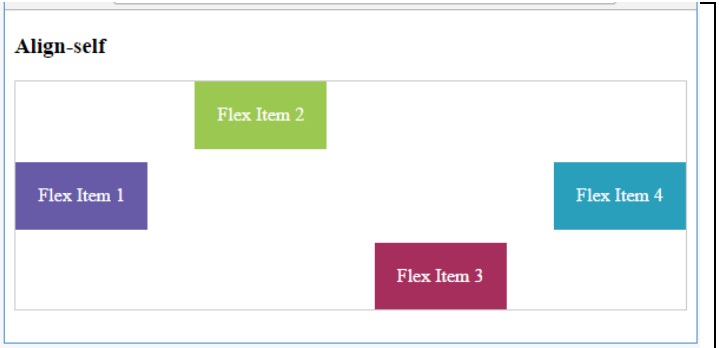
Center

Свойство align-self

Свойство **align-self** позволяет переопределить значение свойства `align-items` для одного элемента. Оно может принимать те же значения плюс зн-ие "auto":

- **auto**: значение по умолчанию, при котором элемент получает значение от свойства `align-items`, которое определено в flex-контейнере. Если в контейнере такой стиль не определен, то применяется значение `stretch`.
- **stretch**
- **flex-start**
- **flex-end**
- **center**
- **baseline**

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>
            .flex-container {
                display: flex;
                border:1px #ccc solid;
                justify-content: space-between;
                align-items: stretch;
                height:12em;
            }
            .flex-item {
                text-align:center;
                font-size: 1em;
                padding: 1.2em;
                color: white;
            }
            .item1 {background-color: #675BA7; align-self: center; }
            .item2 {background-color: #9BC850; align-self: flex-start;}
            .item3 {background-color: #A62E5C; align-self: flex-end;}
            .item4 {background-color: #2A9FBC; align-self: center;}
        </style>
    </head>
```



Здесь для flex-контейнера задано растяжение по высоте с помощью значения `align-items: stretch;`. Однако каждый из элементов переопределяет это поведение

```
26      <body>
27          <h3>Align-self</h3>
28          <div class="flex-container">
29              <div class="flex-item item1">Flex Item 1</div>
30              <div class="flex-item item2">Flex Item 2</div>
31              <div class="flex-item item3">Flex Item 3</div>
32              <div class="flex-item item4">Flex Item 4</div>
33          </div>
34  </html>
```

Выравнивание строк и столбцов. align-content

Свойство **align-content** управляет выравниванием рядов (строк и столбцов) во flex-контейнере и поэтому применяется, если свойство `flex-wrap` имеет значение `wrap` или `wrap-reverse`. Свойство align-content может иметь следующие значения:

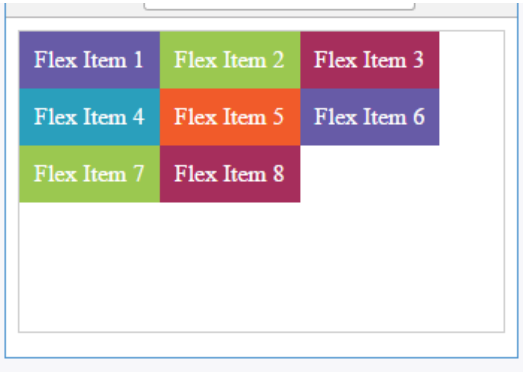
- **stretch**: значение по умолчанию, при котором строки (столбцы) растягиваются, занимая все свободное место
- **flex-start**: строки (столбцы) выравниваются по началу контейнера (для строк - это верхний край, для столбцов - это левый край контейнера)

- **flex-end**: строки (столбцы) выравниваются по концу контейнера (строки - по нижнему краю, столбцы - по правому краю)
- **center**: строки (столбцы) позиционируются по центру контейнера
- **space-between**: строки (столбцы) равномерно распределяются по контейнеру, а между ними образуются одинаковые отступы. Если же имеющегося в контейнере места недостаточно, то действует аналогично значению `flex-start`
- **space-around**: строки (столбцы) равным образом распределяют пространство контейнера, а растояние между первой и последней строкой (столбцом) и границами контейнера составляет половину расстояния между соседними строками (столбцами).

Стоит учитывать, что это свойство имеет смысл, если в контейнере две и больше строки (столбца).

Например, расположение строк в начале контейнера:

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>
            .flex-container {
                display: flex;
                border:1px #ccc solid;
                flex-wrap: wrap;
                height:200px;
                align-content: flex-start;
            }
            .flex-item {
                text-align:center;
                font-size: 16px;
                padding: 10px;
                color: white;
            }
            .item1 {background-color: #675BA7;}
            .item2 {background-color: #9BC850;}
            .item3 {background-color: #A62E5C;}
            .item4 {background-color: #2A9FBC;}
            .item5 {background-color: #F15B2A;}
        </style>
    </head>
```

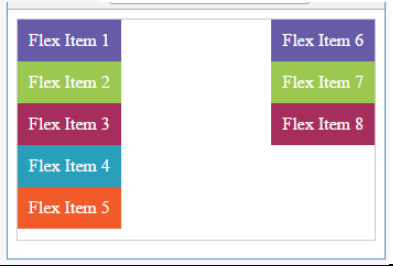


```html
27    <body>
28        <div class="flex-container">
29            <div class="flex-item item1">Flex Item 1</div>
30            <div class="flex-item item2">Flex Item 2</div>
31            <div class="flex-item item3">Flex Item 3</div>
32            <div class="flex-item item4">Flex Item 4</div>
33            <div class="flex-item item5">Flex Item 5</div>
34            <div class="flex-item item1">Flex Item 6</div>
35            <div class="flex-item item2">Flex Item 7</div>
36            <div class="flex-item item3">Flex Item 8</div>
37        </div>
38    </html>
```

Изменим стиль контейнера:

```css
1  .flex-container {
2      display: flex;
3      border:1px #ccc solid;
4      flex-wrap: wrap;
5      height:200px;
6      align-content: space-between;
7      flex-direction: column;
8  }
```

И в этом случае мы получим ряд столбцов, разделенных отступами:



Управление элементами. flex-basis, flex-shrink и flex-grow

Кроме свойств, устанавливающих выравнивание элементов относительно границ flex-контейнера, есть еще три свойства, которые позволяют управлять элементами:

- **flex-basis**: определяет начальный размер flex-элемента
- **flex-shrink**: определяет, как flex-элемент будет уменьшаться относительно других flex-элементов во flex-контейнере
- **flex-grow**: определяет, как flex-элемент будет увеличиваться относительно других flex-элементов во flex-контейнере

flex-basis

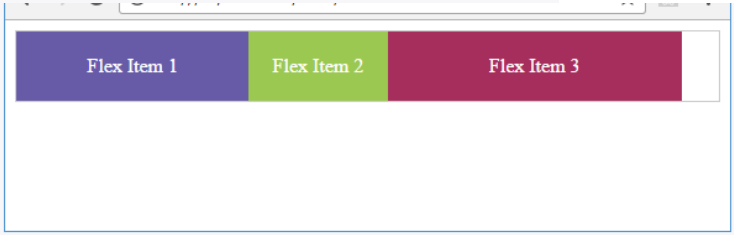
Flex-контейнер может увеличиваться или уменьшаться вдоль своей центральной оси, например, при изменении размеров браузера, если контейнер имеет нефиксированные размеры. И вместе с контейнером также могут увеличиваться и уменьшаться его flex-элементы. Свойство **flex-basis** определяет начальный размер flex-элемента до того, как он начнет изменять размер, подстраиваясь под размеры flex-контейнера.

Это свойство может принимать следующие значения:

- **auto**: начальный размер flex-элемента устанавливается автоматически
- **content**: размер flex-элемента определяется по его содержимому, в то же время это значение поддерживается не всеми современными браузерами, поэтому его пока стоит избегать
- **числовое значение**: мы можем установить конкретное числовое значение для размеров элемента

Например:

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>
            .flex-container {
                display: flex;
                border:1px #ccc solid;
            }
            .flex-item {
                text-align:center;
                font-size: 1em;
                padding: 1.2em;
                color: white;
            }
            .item1 {background-color: #675BA7; flex-basis: auto; width:150px;}
            .item2 {background-color: #9BC850; flex-basis: auto; width:auto;}
            .item3 {background-color: #A62E5C; flex-basis: 200px;width:150px;}
        </style>
    </head>
```



```
22      <body>
23          <div class="flex-container">
24              <div class="flex-item item1">Flex Item 1</div>
25              <div class="flex-item item2">Flex Item 2</div>
26              <div class="flex-item item3">Flex Item 3</div>
27          </div>
28  </html>
```

У первого элемента у свойства `flex-basis` установлено значение auto. Поэтому первый элемент в качестве реального значения для ширины будет использовать значение свойства `width`.

У второго элемента у свойства `flex-basis` установлено значение auto, однако и свойство `width` имеет значение auto. Поэтому реальная ширина элемента будет устанавливаться по его содержимому.

У третьего элемента свойство `flex-basis` имеет конкретное значение, которое и используется. А свойство `width` в этом случае уже не играет никакой роли.

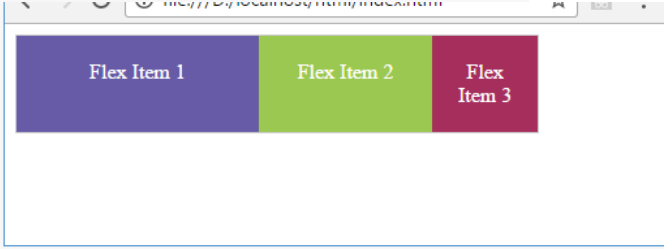
flex-shrink

Если flex-контейнер имеет недостаточно места для размещения элемента, то дальнейшее поведение этого элемента мы можем определить с помощью свойства **flex-shrink**. Оно указывает, как элемент будет усекаться относительно других элементов.

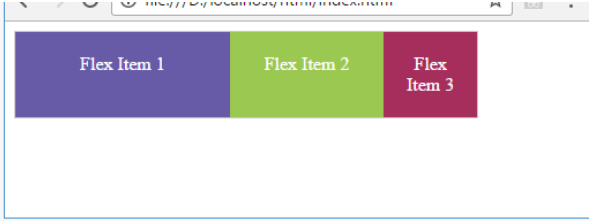
В качестве значения свойство принимает число. По умолчанию его значение 1.

Рассмотрим действие этого свойства на примере:

```html
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>
            .flex-container {
                display: flex;
                border:1px #ccc solid;
                width:400px;
            }
            .flex-item {
                text-align:center;
                font-size: 1em;
                padding: 1.2em;
                color: white;
            }
            .item1 {background-color: #675BA7; flex-basis: 200px; flex-shrink:1;}
            .item2 {background-color: #9BC850; flex-basis: 200px; flex-shrink:2;}
            .item3 {background-color: #A62E5C; flex-basis: 200px; flex-shrink:3;}
        </style>
    </head>
```



```
23      <body>
24          <div class="flex-container">
25              <div class="flex-item item1">Flex Item 1</div>
26              <div class="flex-item item2">Flex Item 2</div>
27              <div class="flex-item item3">Flex Item 3</div>
28          </div>
29  </html>
```



В данном случае начальная ширина каждого элемента равна 200px, то есть совокупная ширина составляет 600px. Однако ширина flex-контейнера составляет всего 400px. То есть размер контейнера недостаточен для вмещения в него элементов, поэтому в действие вступает свойство `flex-shrink`, которое определено у элементов.

Для усечения элементов браузер вычисляет коэффициент усечения (shrinkage factor). Он вычисляется путем перемножения значения свойства `flex-basis` на `flex-shrink`. Таким образом, для трех элементов получим следующие вычисления:

```
// первый элемент
200px * 1 = 200
// второй элемент
200px * 2 = 400
// третий элемент
200px * 3 = 600
```

В итоге получаем, что для второго элемента коэффициент усечения в два раза больше, чем коэффициент для первого элемента. А для третьего элемента коэффициент больше в три раза, чем у первого элемента. Поэтому в итоге первый элемент при усечении будет в три раза больше, чем третий и в два раза больше, чем второй.

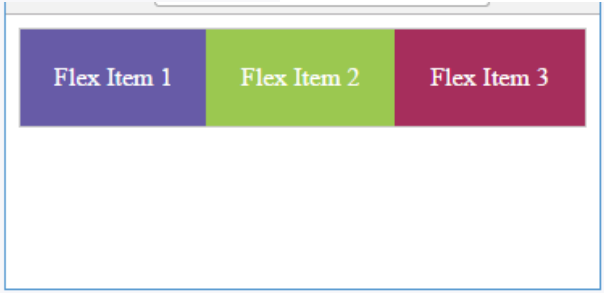
flex-grow

Свойство **flex-grow** управляет расширением элементов, если во flex-контейнере есть дополнительное место. Данное свойство во многом похоже на свойство `flex-shrink` за тем исключением, что работает в сторону увеличения элементов.

В качестве значения свойство flex-grow принимает положительное число, которое указывает, во сколько раз элемент будет увеличиваться относительно других элементов при увеличении размеров flex-контейнера. По умолчанию свойство flex-grow равно 0.

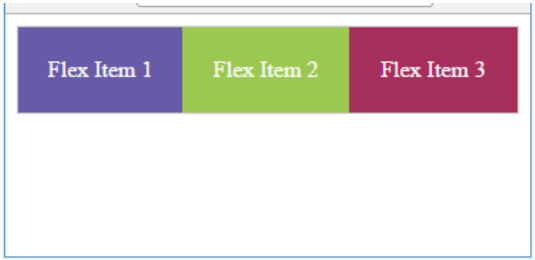
Итак, используем свойство flex-grow:

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>
            .flex-container {
                display: flex;
                border:1px #ccc solid;
            }
            .flex-item {
                text-align:center;
                font-size: 1em;
                padding: 1.3em;
                color: white;
            }
            .item1 {background-color: #675BA7; flex-grow:0;}
            .item2 {background-color: #9BC850; flex-grow:1;}
            .item3 {background-color: #A62E5C; flex-grow:2;}
        </style>
    </head>
```

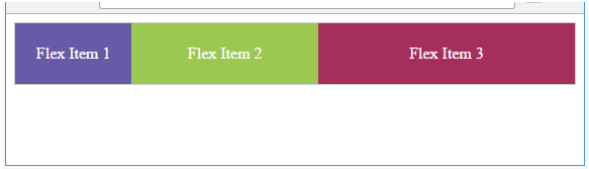


```html
22      <body>
23          <div class="flex-container">
24              <div class="flex-item item1">Flex Item 1</div>
25              <div class="flex-item item2">Flex Item 2</div>
26              <div class="flex-item item3">Flex Item 3</div>
27          </div>
28  </html>
```

Итак, для каждого элемента есть базовые начальные размеры. Здесь явным образом размеры для элементов не указаны, поэтому размер каждого элемента в данном случае будет складываться из размеров внутреннего содержимого, к которым добавляются внутренние отступы.



По мере растягивания контейнера будут увеличиваться элементы в соответствии со свойством flex-grow, которое указано для каждого элемента. Пространство, на которое растягивается контейнер, считается дополнительным пространством.



Так как у первого элемента свойство flex-grow равно 0, то первый элемент будет иметь константные постоянные размеры. У второго элемента flex-grow равно 1, а третьего - 2.

Таким образом, в сумме они дадут 0 + 1 + 2 = 3. Поэтому второй элемент будет увеличиваться на 1/3 дополнительного пространства, на которое растягивается контейнер, а третий элемент будет получать 2/3 дополнительного пространства.

Свойство flex

Свойство **flex** является объединением свойств `flex-basis`, `flex-shrink` и `flex-grow` и имеет следующий формальный синтаксис:

```
flex: [flex-grow] [flex-shrink] [flex-basis]
```

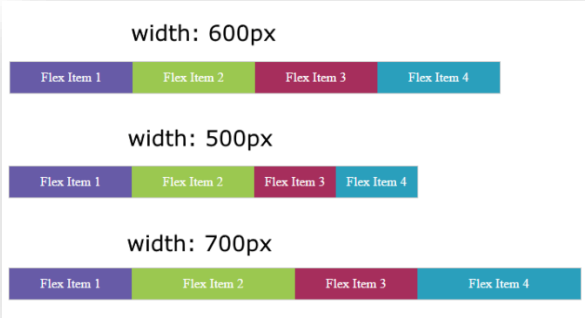
По умолчанию свойство `flex` имеет значение `0 1 auto`.

Кроме конкретных значений для каждого из подсвойств мы можем задать для свойства flex одно из трех общих значений:

- `flex: none`: эквивалентно значению `0 0 auto`, при котором flex-элемент не растягивается и не усекается при увеличении и уменьшении контейнера
- `flex: auto`: эквивалентно значению `1 1 auto`
- `flex: initial`: эквивалентно значению `0 1 auto`

Так, применим свойство flex:

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>
            .flex-container {
                display: flex;
                border:1px #ccc solid;
                width: 600px;
            }
            .flex-item {
                text-align:center;
                font-size: 16px;
                padding: 10px 0;
                color: white;
            }
            .item1 {background-color: #675BA7; width: 150px; flex: 0 0 auto }
            .item2 {background-color: #9BC850; width: 150px; flex: 1 0 auto;}
            .item3 {background-color: #A62E5C; width: 150px; flex: 0 1 auto;}
            .item4 {background-color: #2A9FBC; width: 150px; flex: 1 1 auto;}
        </style>
    </head>
```



```html
    <body>
        <div class="flex-container">
            <div class="flex-item item1">Flex Item 1</div>
            <div class="flex-item item2">Flex Item 2</div>
            <div class="flex-item item3">Flex Item 3</div>
            <div class="flex-item item4">Flex Item 4</div>
        </div>
</html>
```

Здесь каждый элемент имеет начальную ширину в 150 пикселей, так как у всех элементов свойство `flex-basis` имеет значение 0, что в целом для всех элементов будет составлять 600 пикселей.

При сжатии контейнера будут уменьшаться 3-й и 4-й элементы, так как у них свойство `flex-shrink` больше нуля. И так как у обоих элементов это свойство равно 1, то оба элемента будут уменьшаться в равных долях.

При растяжении контейнера будут увеличиваться 2-й и 4-й элементы, так как у этих элементов свойство `flex-grow` больше нуля. И также, так как это свойство равно 1, то эти элементы будут увеличиваться в равных долях.

Многоколоночный дизайн на Flexbox

Теперь рассмотрим, как мы можем сделать простейшие многоколонночные макеты страницы с помощью Flexbox.

Двуколоночный дизайн

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>
            *{
                box-sizing: border-box;
            }
            html,
            body {
                padding: 0;
                margin: 0;
                font-family: verdana, arial, sans-serif;
            }

            body {
                display: flex;
                padding: 1em;
                flex-direction: column;
            }

            .item {
                background-color: #455a64;
                color: #fff;
                font-size: 1.1em;
                padding: 1em;
            }

            .item:nth-child(even) {
                background-color: #607d8b;
            }
            @media screen and (min-width: 600px) {

                body {
                    flex-direction: row;
                }
            }
        </style>
    </head>
```

Здесь flex-контейнером является элемент body. Так как на мобильных устройствах (особенно смартфонах) размер экрана не такой большой, поэтому по умолчанию устанавливаем расположение элементов в столбик.

Однако для устройств с экраном от 600px и выше действует правило media-query, которое устанавливает расположение в виде строки.

```
41    <body>
42        <div class="item">
43            <h2>Что такое Lorem Ipsum?</h2>
44            <p>Lorem Ipsum - это текст-"рыба", часто используемый в печати и вэб-дизайне. Lorem Ipsum является
45            стандартной "рыбой" для текстов на латинице с начала XVI века...</p>
46        </div>
47
48        <div class="item">
49            <h2>Откуда он появился?</h2>
50            <p>Многие думают, что Lorem Ipsum - взятый с потолка псевдо-латинский набор слов, но это не совсем так.
51            Его корни уходят в один фрагмент классической латыни 45 года н.э...</p>
52        </div>
53    </body>
54 </html>
```

Трехколоночный режим

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>
            *{
                box-sizing: border-box;
            }
            html,
            body {
                padding: 0;
                margin: 0;
                font-family: verdana, arial, sans-serif;
            }

            body {
                display: flex;
                padding: 1em;
                flex-direction: column;
            }

            .item {
                background-color: #455a64;
                color: #fff;
                font-size: 1.1em;
                padding: 1em;
                flex: 1;
            }

            .item:nth-child(1) {
                background-color: #607d8b;
            }
            @media screen and (min-width: 600px) {

                body {
                    flex-direction: row;
                }
                .item:nth-child(2) {
                    order: -1;
                }
            }
        </style>
    </head>
```

В отличие от предыдущего примера здесь добавлен еще один элемент. Особенностью этого примера является то, что столбцы имеют одинаковые размеры. Для этого у них установлено свойство `flex: 1`, то есть при растяжении или уменьшении границ контейнера все элементы будут масштабироваться на равную величину.

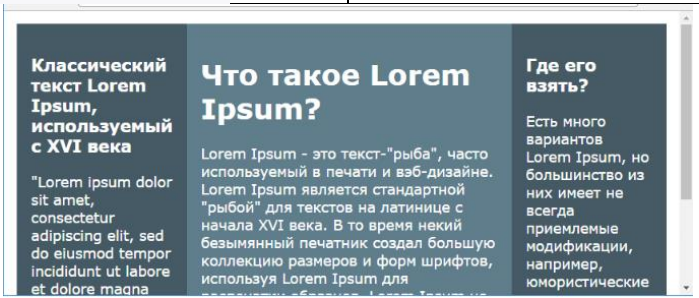
И кроме того, при ширине экрана больше 600px у второго элемента устанавливается свойство `order: -1`, благодаря чему этот элемент помещается первым:

```
45      <body>
46
47          <div class="item">
48              <h1>Что такое Lorem Ipsum?</h1>
49              <p>Lorem Ipsum - это текст-"рыба", часто используемый в печати и вэб-дизайне. Lorem Ipsum является стандартной "рыбой"
50              для текстов на латинице с начала XVI века. В то время некий безымянный печатник создал ...</p>
51          </div>
52
53          <div class="item">
54              <h3>Классический текст Lorem Ipsum, используемый с XVI века</h3>
55              <p>"Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore
56              magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco</p>
57          </div>
58          <div class="item">
59              <h3>Где его взять?</h3>
60              <p>Есть много вариантов Lorem Ipsum, но большинство из них имеет не всегда приемлемые модификации, например,
61              юмористические вставки или слова, которые даже отдалённо не напоминают латынь.</p>
62          </div>
63      </body>
64  </html>
```

Подобным образом можем добавить и большее количество столбцов. Но в данном случае по умолчанию столбцы имеют одинаковую ширину. Но что делать, если один из столбцов (как правило, центральный) должен иметь ширину больше, чем у остальных? Для этого добавим в стили страницы следующее правило:

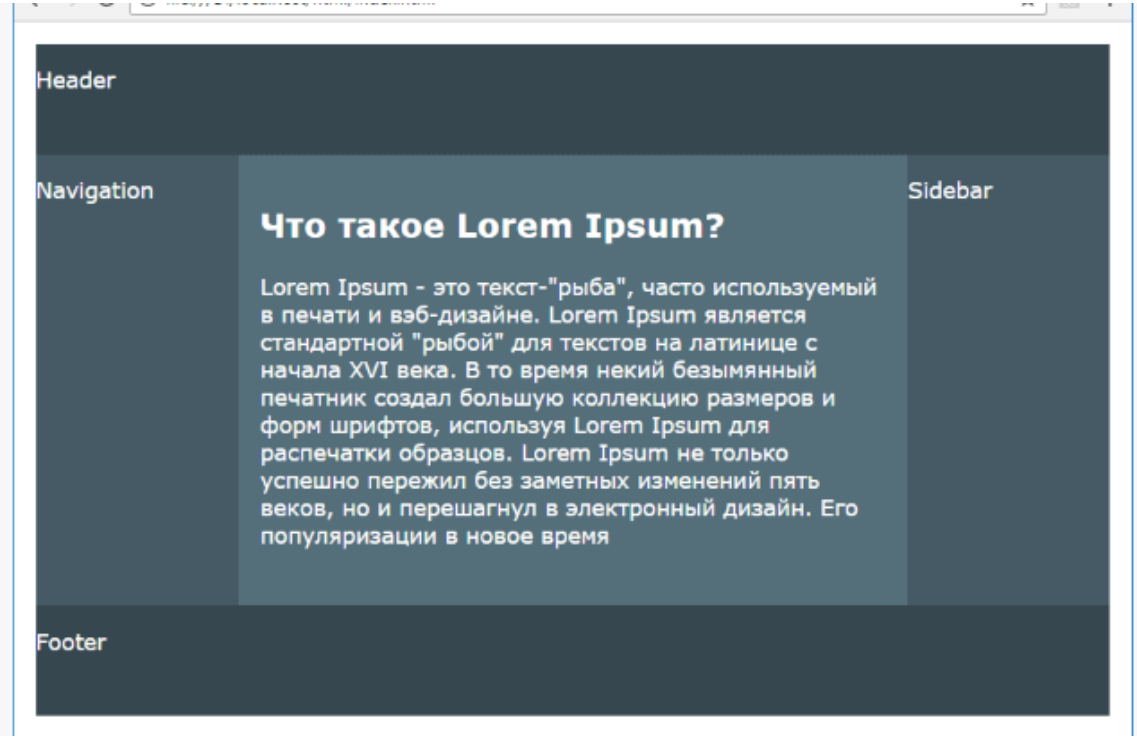
```
.item:first-child {
    flex: 0 0 50%;
}
```

В этом случае первый элемент всегда будет занимать 50% пространства контейнера



Макет страницы на Flexbox

Теперь рассмотрим создание стандартного макета страницы, который состоит их шапки, футера и центральной части, в которой есть три столбца: основное содержимое и два сайдбара.

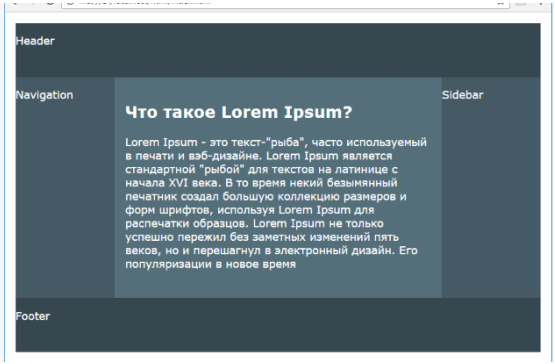


Для этого определим следующую веб-страницу:

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>Flexbox в CSS3</title>
        <style>
            *{
                box-sizing: border-box;
            }
            html,
            body {
                padding: 0;
                margin: 0;
                font-family: verdana, arial, sans-serif;
            }

            body {
                display: flex;
                padding: 1em;
                flex-direction: column;
            }
```

```
22
23              .item {
24                  background-color: #455a64;
25                  color: #fff;
26                  font-size: 1.1em;
27                  padding: 1em;
28                  flex: 1;
29              }
30
31              .item:nth-child(1) {
32                  background-color: #607d8b;
33              }
34              @media screen and (min-width: 600px) {
35
36                  body {
37                      flex-direction: row;
38                  }
39                  .item:nth-child(2) {
40                      order: -1;
41                  }
42              }
43          </style>
44      </head>
```

```
45      <body>
46
47          <div class="item">
48              <h1>Что такое Lorem Ipsum?</h1>
49              <p>Lorem Ipsum - это текст-"рыба", часто используемый в печати и вэб-дизайне. Lorem Ipsum является стандартной "рыбой"
50              для текстов на латинице с начала XVI века. В то время некий безымянный печатник создал ...</p>
51          </div>
52
53          <div class="item">
54              <h3>Классический текст Lorem Ipsum, используемый с XVI века</h3>
55              <p>"Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore
56              magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco</p>
57          </div>
58          <div class="item">
59              <h3>Где его взять?</h3>
60              <p>Есть много вариантов Lorem Ipsum, но большинство из них имеет не всегда приемлемые модификации, например,
61              юмористические вставки или слова, которые даже отдалённо не напоминают латынь.</p>
62          </div>
63      </body>
64  </html>
```



Итак, flex-контейнером верхнего уровня здесь является элемент body.

Его flex-элементами являются header, main и footer. Body располагает все свои элементы сверху вниз в столбик. Здесь также стоит отметить, что при ширине от 600px и выше для заполнения всего пространства браузера у body устанавливается стиль `height: 100vh;`.

Элементы header и footer аналогичны. Их свойство `flex: 0 0 5em;` указывают, что при любом изменении контейнера эти элементы будут иметь размер в 5em. То есть они имеют статический размер.

Более сложным является элемент main, который определяет основное содержимое. При этом будучи flex-элементом, он также является flex-контейнером для вложенных элементов и управляет их позиционированием. При ширине браузера до 600px он располагает элементы в столбик, что очень удобно на мобильных устройствах.

При ширине от 600px вложенные элементы nav, article и aside располагаются в виде строки. И поскольку при такой ширине браузера родительский элемент body заполняет по высоте все пространство браузера, то для заполнения всей высоты контейнера body при его изменении у элемента main устанавливается свойство `flex: 1 1 auto;`.

У вложенных в main flex-элементов стоит отметить, что элемент навигации nav и элемент сайдбара aside будут иметь одинаковые размеры при масштабировании контейнера. А элемент article, содержащий основное содержимое, будет соответственно больше. При этом хотя nav определен после элемента article, но благодаря установке свойства `order: -1` блок навигации будет стоять до блока article.